



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE

CI EXPERT - TRILHA RTL SYNTHESIS

Projeto 1

Vending Machine

Gabriel Florentino

Campina Grande, 2026

Sumário

1	Introdução	1
1.1	Especificação Funcional	1
1.2	Especificação Estrutural	2
2	Desenvolvimento	3
2.1	Módulo control_unit	3
2.2	Módulo cash_register	5
2.3	Módulo comparator	5
2.4	Módulo subtractor	5
2.5	Módulo stock_memory	6
3	Verificação	7
3.1	Compra bem-sucedida com troco	7
3.2	Credito insuficiente	8
3.3	Cancelamento	9
3.4	Estoque zerado	10
4	Síntese	12
5	Análise	13
6	Conclusão	15

1 Introdução

A proposta deste relatório é apresentar e documentar todo o processo de desenvolvimento e verificação do projeto de uma *Vending Machine*, utilizando a linguagem de programação SystemVerilog e as ferramentas de síntese lógica da Synopsys. *Vending machine* é, em tradução direta, uma máquina de vendas autônoma na qual é possível depositar moedas e, em troca, selecionar um produto cujo preço seja menor ou igual ao valor de moedas adicionado. O projeto envolveu o desenvolvimento do controlador central desta máquina, cujas especificações são descritas a seguir.

1.1 Especificação Funcional

O controlador irá gerenciar uma *Vending Machine* que contém 4 produtos: café, água, suco e salgados. O usuário irá depositar as moedas na máquina, uma por uma, e após uma confirmação irá selecionar o produto. O controlador então verifica na memória o preço e o estoque do produto selecionado, e se a soma das moedas for maior ou igual ao preço do produto, a máquina irá liberar o produto e o troco para o usuário. O usuário tem a opção de cancelar a compra a qualquer momento. Caso isto aconteça, a máquina irá devolver o valor adicionado ao usuário e voltará para o seu estado de espera. Na ocasião de algum erro, seja ele da máquina ou do procedimento, máquina também devolve o valor adicionado de volta ao usuário.

1.2 Especificação Estrutural

O controlador terá como entrada 6 portas, sendo elas:

- **clk** - 1 bit - Clock principal da máquina.
- **rst** - 1 bit - Sinal de reset.
- **coin_in** - 2 bits - Sinal que representa a moeda adicionada/
- **confirm** - 1 bit - Sinal de confirmação da compra.
- **cancel** - 1 bit - Sinal de cancelamento da compra.

E terá como saída, as seguintes portas:

- **dispense** - 1 bit - Sinal de liberação do produto.
- **change_out** - 8 bits - Valor do troco liberado após a compra.
- **error** - 1 bit - Sinal que indica que a máquina entrou em um estado de erro.
- **display** - 8 bits - Valor do crédito adicionado para ser mostrado no display.
- **state_out** - 3 bits - Estado atual da FSM que representa a operação da máquina.

O controlador será implementado com 5 módulos funcionais:

- **cash_register** - Módulo responsável por armazenar o saldo do usuário.
- **comparator** - Módulo responsável pela verificação do crédito em relação ao preço e pelo controle de estoque.
- **stock_memory** - Memória dos produtos. Guarda o preço e o estoque de cada um dos produtos.
- **subtractor** - Módulo que calcula o troco gerado após a compra.
- **control_unit** - Módulo responsável pela orquestração da operação da máquina. Implementa uma *Finite State Machine*.

2 Desenvolvimento

Nesta seção vão ser apresentados os módulos desenvolvidos durante a realização do projeto. O Módulo *Top Level* **vending_top** representa o controlador com os 5 submódulos dentro dele. O módulo vending top é apresentado na figura 2.1

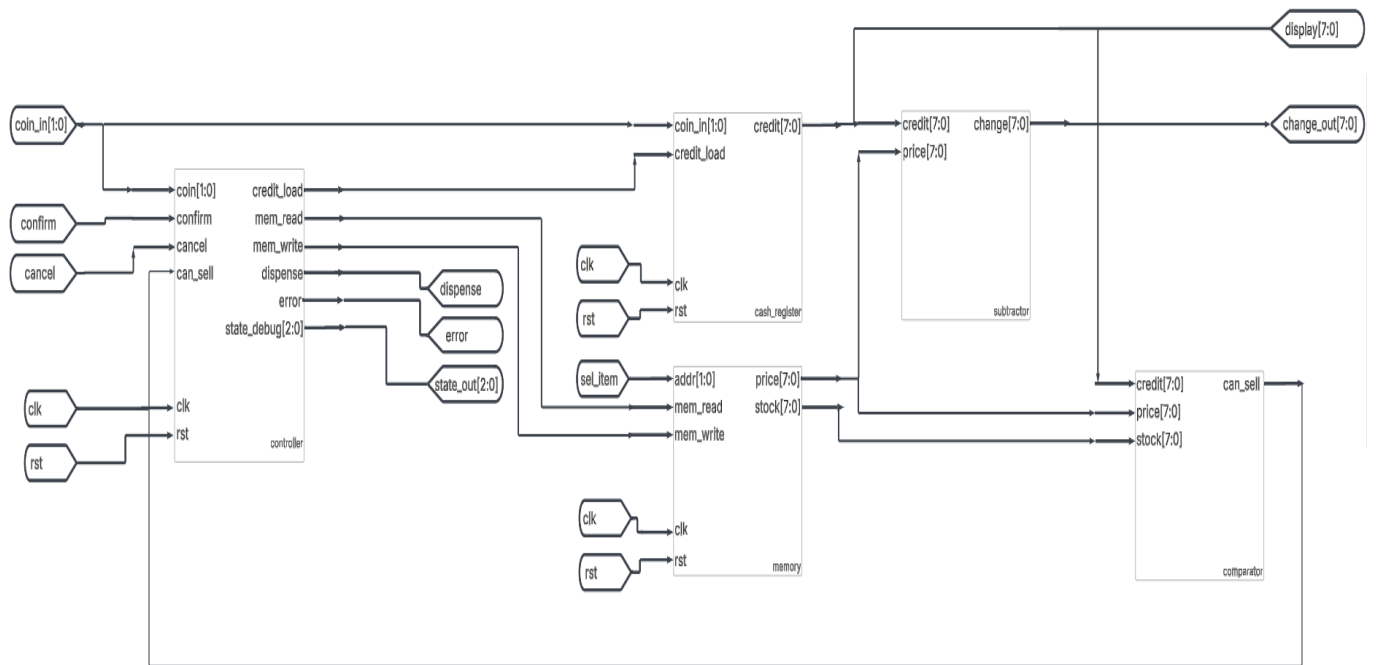


Figura 2.1: Módulo Top Level - Vending Top

2.1 Módulo control_unit

Este é o módulo principal do controlador, ele implementa uma máquina *Moore* com 6 estados. Cada um desses estados é responsável por verificar as saídas dos outros submódulos, para determinar a transição corretamente. O diagrama de estados pode ser observado na

figura2.2

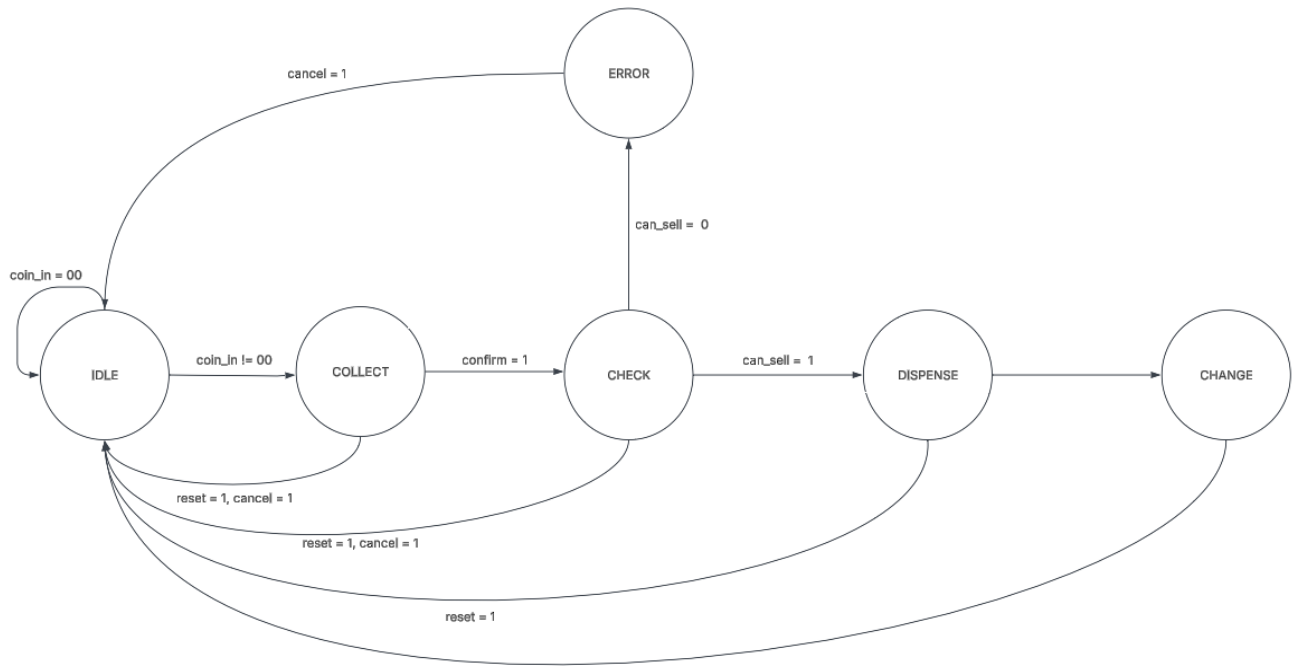


Figura 2.2: Diagrama de estado - control_unit

Cada estado representa uma etapa na compra de um produto da máquina:

- IDLE: Estado de espera. Permanece nele enquanto coin_in é igual a zero, caso contrário passa para o estado de coleta.
- COLLECT: Estado de coleta de moedas. Após o sinal de confirmação ser ativado, passa para o estado de checagem dos produtos.
- CHECK: Estado de checagem dos produtos. Nele, o controlador ativa o sinal de leitura da memória e espera pelo resultado do comparador, para saber se passa para o estado de erro, caso o credito seja insuficiente para comprar o produto ou que não tenha estoque, ou se passa para o estado de liberação.
- DISPENSE: Estado de liberação do produto. Neste estado o controlador faz a liberação

do produto e diminui uma unidade de produto comprado. Após um ciclo de clock passa para o estado de troco.

- **CHANGE:** Estado de troco. É neste estado onde o crédito é devolvido ao usuário e onde o crédito acumulado é zerado. Após um ciclo de clock volta ao estado de espera.
- **ERROR:** Estado de erro. Aqui o controlador também estorna o valor do crédito acumulado, porém permanece neste estado até que o sinal de cancel seja ativado.

2.2 Módulo cash_register

Neste módulo é onde fica armazenado o crédito depositado pelo usuário. É um registrador síncrono de 8 bits responsável por acumular o valor de todas as moedas inseridas pelo usuário, em centavos. O este módulo só acumula o valor das moedas quando o sinal de credit_load está ativo. A o valor coin_in é uma representação binária de 2 bits de uma moeda, podendo assumir os seguintes valores:

- | | |
|--------------------|--------------------|
| • 2'b00 -> R\$0,00 | • 2'b11 -> R\$0,50 |
| • 2'b01 -> R\$0,25 | • 2'b11 -> R\$1,00 |

2.3 Módulo comparator

Este módulo é puramente combinacional, sempre avaliando se o crédito é maior que o preço e se o estoque é maior que zero. Se estas duas condições são satisfeitas, o sinal can_sell é ativado, caso contrário o sinal permanece desativado.

2.4 Módulo subtractor

Também é puramente combinacional, porém, sempre calcula a diferença entre o valor do crédito menos o preço. Caso o preço seja maior que o crédito, só retornará o valor do crédito depositado.

2.5 Módulo stock_memory

Este módulo é responsável pelo armazenamento dos preços e estoque dos produtos. Cada endereço da memória representa um dos produtos à venda. Quando mem_read é ativado, ele irá buscar o produto e no próximo ciclo de clock deixará disponível o preço do produto, e a quantidade em estoque. Ambos os valores ficarão disponíveis nas saídas price e stock. Quando mem_write é ativado o módulo irá subtrair 1 do valor do estoque atual. A tabela a seguir 2.1 mostra o endereço, preço e estoque definido para cada produto

Endereço	Preço	Estoque	Produto
0x0	R\$0,25	5	Cáfe
0x1	R\$0,50	5	Água
0x2	R\$0,75	3	Suco
0x3	R\$1,00	2	Salgado

Tabela 2.1: Endereços de cada produto

3 Verificação

A verificação do projeto foi realizada utilizando os softwares **vlogan** e **VCS**, ambas da desenvolvidas pela empresa *Synopsys*. Para cada do projeto módulo existe um *testchbench* para verificação das funcionalidades individualmente. O *testchbench* principal, chamado *vending_top_tb*, é o responsável por simular o comportamento da *Vending machine* de forma integral. Para que a cobertura do *testchbench* fosse abrangente, foram escolhidos quatro cenários de funcionamento da máquina. A seguir, cada cenário é descrito juntamente com o respectivo resultado do teste de verificação e seu diagrama temporal de simulação.

3.1 Compra bem-sucedida com troco

Neste *testchbench* uma moeda de R\$1,00 vai ser inserida na máquina e o produto selecionado será um café. No final, é esperado que a máquina libere o produto e que um troco de 75 centavos seja devolvido ao usuário.

```
-----  
Purchase successful  
-----  
Dispense Test result: PASS, expected = 1, actual 1  
Change Out Test result: PASS, expected = 75, actual 75  
Credit Test result: PASS, expected = 0, actual 0
```

Figura 3.1: TB compra bem-sucedida com troco - saída

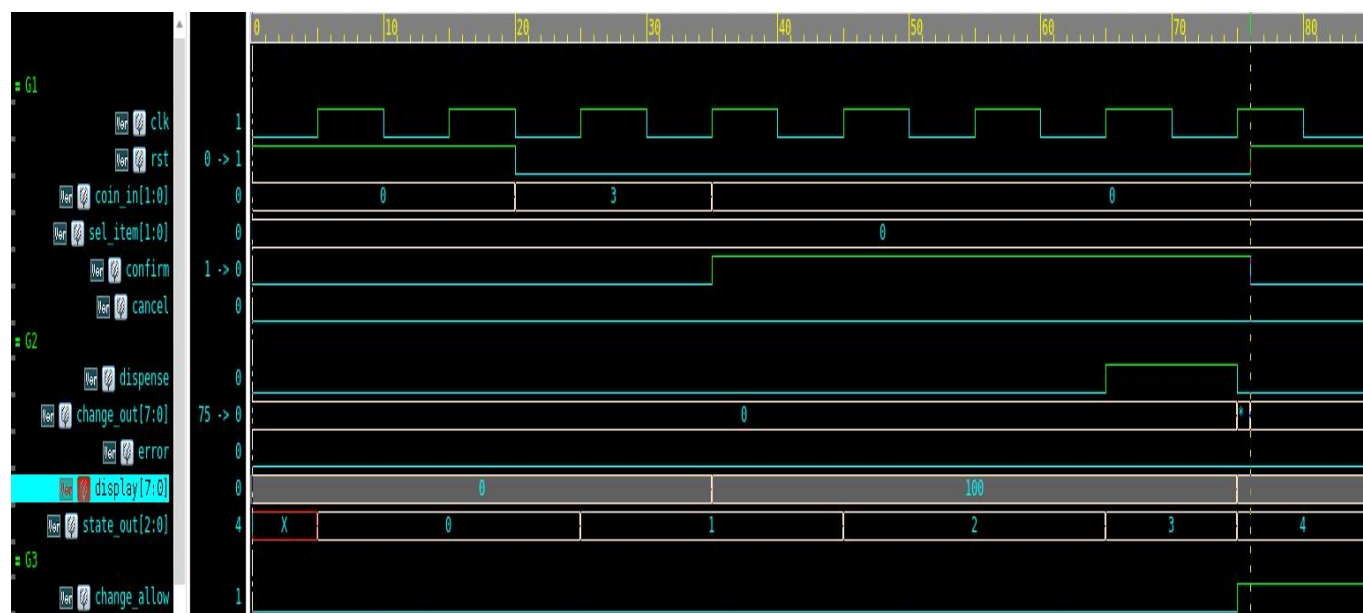


Figura 3.2: TB compra bem-sucedida com troco - simulação

3.2 Credito insuficiente

Aqui uma moeda de R\$0,25 vai ser inserida e o produto selecionado será um salgado. Já que o preço do salgado é maior que o crédito adicionado, a flag de erro será ativada, e a máquina devolverá o valor inserido por meio do troco.

```

-----
Insuficient Funds
-----
Error Test result: PASS, expected = 1, actual 1
Change out Test result: PASS, expected = 50, actual 50

```

Figura 3.3: TB crédito insuficiente - saída

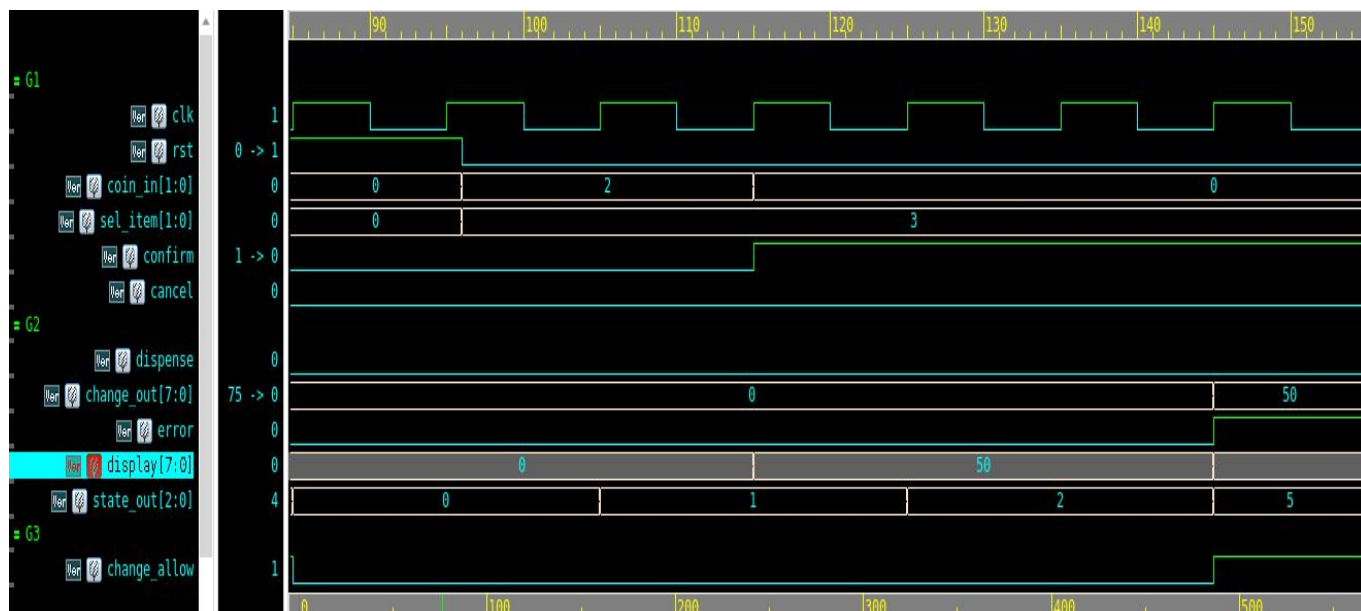


Figura 3.4: TB crédito insuficiente - simulação

3.3 Cancelamento

No TB de cancelamento, duas moedas de R\$1,00 serão inseridas. Porém, antes de confirmar, a flag de cancelamento vai ser ativada. A máquina vai então retornar o valor ao usuário e voltará para o seu estado de espera.

```

-----
Purchase cancel
-----
Credit Test result: PASS, expected = 0, actual 0
Change out Test result: PASS, expected = 200, actual 200

```

Figura 3.5: TB cancelamento - saída

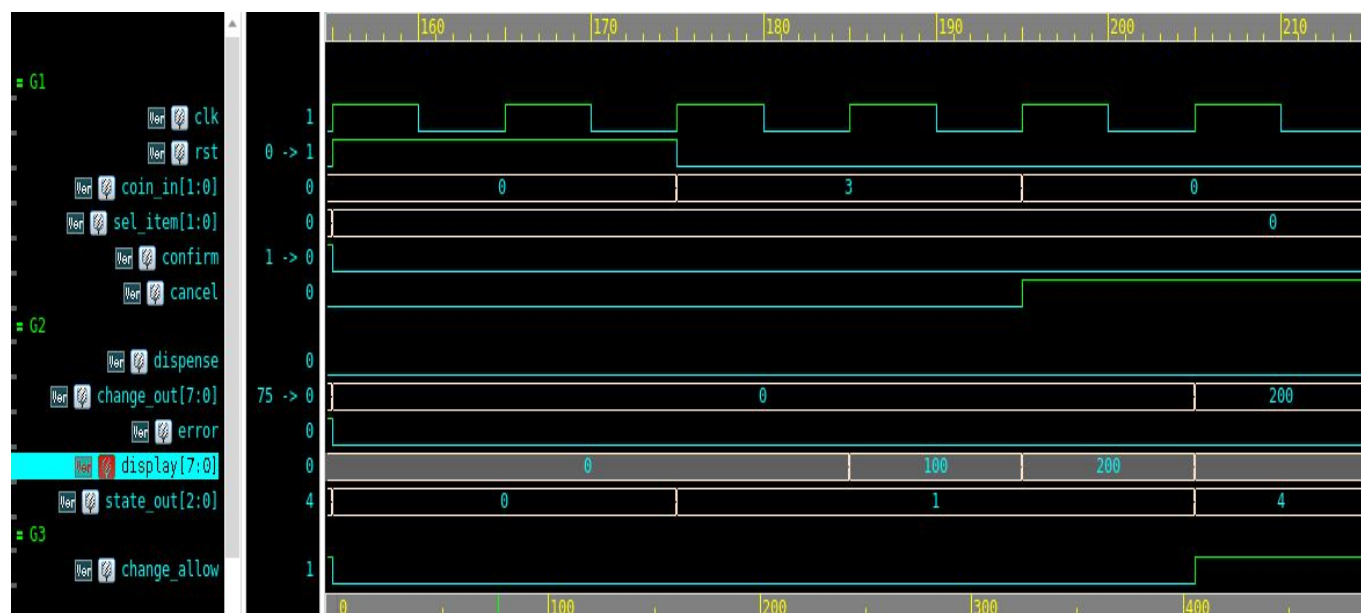


Figura 3.6: TB cancelamento - simulação

3.4 Estoque zerado

Neste TB são efetuadas 6 compras de cafe. Como o estoque de café é de 5 unidades, quando a tentativa de compra da 6 unidade for executada, a máquina detectará que não existe mais estoque e ativará a flag de erro.

Stock Empty			
Error in 1	Purchase Test result: PASS,	expected = 0,	actual 0
Error in 2	Purchase Test result: PASS,	expected = 0,	actual 0
Error in 3	Purchase Test result: PASS,	expected = 0,	actual 0
Error in 4	Purchase Test result: PASS,	expected = 0,	actual 0
Error in 5	Purchase Test result: PASS,	expected = 0,	actual 0
Error in 6	Purchase Test result: PASS,	expected = 1,	actual 1

Figura 3.7: TB estoque zerado - saída

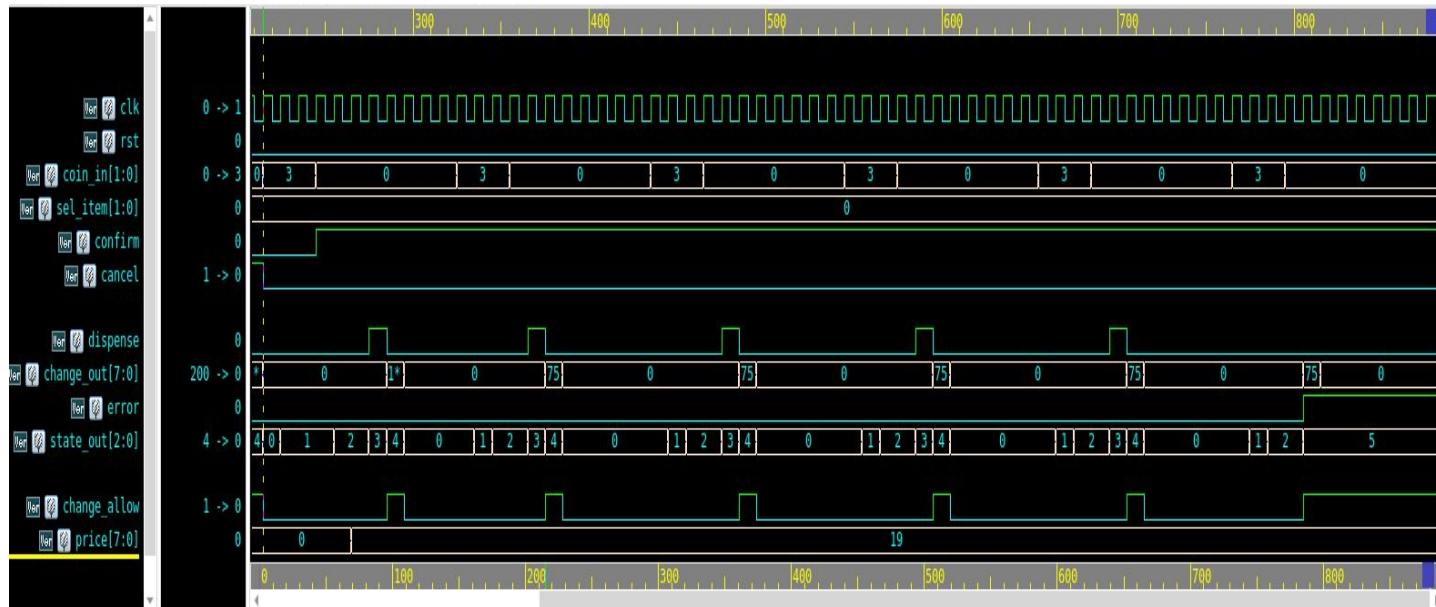


Figura 3.8: TB estoque zerado - simulação

4 Síntese

Após os testes de verificação, foi dado início a etapa de síntese lógica. Nesta etapa a ferramenta *Design Compiler*, também da empresa *Synopsys*, foi utilizada. Para esse projeto a biblioteca (PDK) escolhida foi `saed32rvt_tt1p05v25c.db`. Aqui, um script TCL foi criado para que o processo de síntese lógica fosse feito de forma automática. Além do script de síntese, foi criado um arquivo de *constraints* com as seguintes restrições:

- Clock de 20 ns
- Clock uncertainty de 0.5 ns para modelar o jitter e skew
- Input delay de 3 ns
- Capacitância de saída de 0.05 pf

Para que fosse possível avaliar os limites do design, foi avaliado quais seriam os valores de *slack* e área para cada valor de clock, saindo de 20 até 10 ns em passos de 2 ns. A tabela a seguir apresenta os valores obtidos durante o experimento: 1 90 9

Período	Frequência	Slack(ns)	Área
20	50 MHz	13.36	976.50
18	55,56 MHz	11.36	976.50
16	62,50 MHz	09.36	976.50
14	71,43 MHz	07.36	976.50
12	83,33 MHz	05.36	976.50
10	100 MHz	03.36	976.50
8	100 MHz	01.36	976.50
6	100 MHz	-0.64	976.50

Tabela 4.1: Valores de Slack e Área para cada período de clock

5 Análise

De acordo com os valores encontrados na tabela4.1, o período de 8 ns é o menor período para qual o slack é não negativo. No período de 6 ns, o slack encontrado é de -0.61 ns e é de 1.36 ns no período de 8 ns. Retirando a opção -noautoangroup, houve alteração nos valores de slack e timing. Os resultados estão descritos na tabela a seguir.

Período	Frequência	Slack(ns)	Área
20	50 MHz	13.17	856.02
18	55,56 MHz	11.17	856.02
16	62,50 MHz	09.17	856.02
14	71,43 MHz	07.17	856.02
12	83,33 MHz	05.17	856.02
10	100 MHz	03.17	856.02
8	100 MHz	01.17	856.02
6	100 MHz	-0.83	856.02

Tabela 5.1: Valores de Slack e Área para após retirar a opção de -noautoangroup

Como é possível observar na tabela, tanto os valores de slack quanto a área diminuíram em comparação à tabela4.1. Um slack menor representa um maior atraso entre a chegada esperada e atual.

O relatório de *timing*, além de sinalizar os valores de slack, mostram qual é o caminho crítico do design. O caminho crítico pode ser entendido como o o caminho de maior slack dentro do design. O seguinte caminho foi reportado como caminho crítico pela ferramenta:

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	3.00	3.00 f
cancel (in)	0.01	3.01 f
U182/Y (INVX0_RVT)	0.04	3.05 r
U208/Y (NAND2X0_RVT)	0.06	3.11 f
U210/Y (INVX0_RVT)	0.05	3.16 r
U224/Y (OA221X1_RVT)	0.06	3.22 r
U225/S0 (HADDX1_RVT)	0.08	3.29 f
U226/Y (AND2X1_RVT)	0.04	3.33 f
change_out[4] (out)	0.00	3.33 f
data arrival time		3.33

Figura 5.1: Caminho crítico detectado

É possível observar que o caminho crítico começa pelo pino de entrada cancel e termina no quarto bit do barramento change_out, responsável pelo troco liberado ao usuário. Pelo o que é observado do caminho crítico, o maior tempo gasto está associado ao estado de erro, já que é quando é necessário utilizar o pino de cancel, e toda a lógica combinacional associada ao troco da compra.

Uma possível medida que poderia ser adotada para reduzir a influência deste bloco combinacional no tempo total seria utilizar uma lógica sequencial para liberação do troco no estado de error. Isso possibilitaria uma redução significativa do atraso do caminho crítico, melhorando o slack do design.

Para que a área fosse otimizada, poderia ser adicionado ao comando de síntese (compile_ultra) a seguinte flag: -area_effort high. Esse comando iria possibilitar que o compilador otimizasse ao máximo a área total do controlador.

6 Conclusão

O objetivo deste relatório foi apresentar todo o processo de desenvolvimento e verificação do controlador de uma Vending Machine em SystemVerilog. O projeto envolveu a criação de um módulo top-level chamado `vending_top`, que foi estruturado com cinco submódulos funcionais dentro dele. O módulo principal é a `control_unit`, que implementa uma máquina Moore com 6 estados para coordenar a operação geral do sistema, trabalhando junto com o `cash_register` para guardar o saldo em centavos, o comparador para avaliar as condições de venda e a `stock_memory` para gerenciar os preços e a quantidade de produtos.

A etapa de verificação foi realizada com os softwares vlogan e VCS. Por meio do testbench `vending_top_tb`, foi possível simular o comportamento da máquina de forma integral, cobrindo quatro cenários diferentes de funcionamento. O circuito funcionou exatamente como esperado na simulação de compra bem-sucedida com troco, na ativação de erro por crédito insuficiente, no cancelamento da compra com devolução do valor inserido e no tratamento de estoque zerado, que ficou comprovado quando a máquina impediu a sexta tentativa de compra de café.

Depois dos testes, foi dada continuidade ao projeto com a síntese lógica na ferramenta Design Compiler, utilizando uma biblioteca de 32 nm. Variando o período do clock no script de restrições, foi possível avaliar os valores de slack e área gerados para o circuito. Quando a opção `-noautoungroup` foi retirada do script, os algoritmos da ferramenta reorganizaram os blocos lógicos, o que causou uma diminuição na área total de 976.50 para 856.02, acompanhada por uma redução no slack. Por fim, a análise do relatório de timing mostrou que o caminho crítico começa no pino de entrada `cancel` e termina no quarto bit do barramento de troco `change_out[4]`, com um tempo de chegada de 3.33 ns. Como lição e sugestão de melhoria, mudar a lógica combinacional do troco no estado de erro para uma lógica sequencial seria a medida adotada ideal para reduzir esse atraso e melhorar o slack do design.